

IoT Based System for Road Maintenance

by

K S Vikash (20BEC1019)

R Siddharth (20BEC1098)

Surjyabho Deb (20BEC1130)

A project report submitted to

Dr. BERLIN HENCY

SCHOOL OF ELECTRONICS ENGINEERING

in partial fulfillment of the requirements for the course of

ECE3502 – IoT Domain Analyst

in

**B.Tech. ELECTRONICS AND COMMUNICATION
ENGINEERING**



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Vandalur – Kelambakkam Road

Chennai – 600127

APRIL 2023

BONAFIDE CERTIFICATE

This is to certify that the Project work titled “IoT Based System For Road Maintenance” is being submitted by **K S Vikash (20BEC1019), R Siddharth (20BEC1098), Surjyabho Deb (20BEC1130)** for the course **ECE 3502-IoT Domain Analyst**, is a record of bonafide work done under my guidance. The contents of this project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University.

Dr. Berlin Hency

Associate Professor

School of Electronics Engineering (SENSE),

VIT University, Chennai

Chennai – 600 127

ABSTRACT

The current state of roads in India is substandard and many problems that arise due to their poor condition, including safety concerns, increased travel time, and higher maintenance costs. The government's inability to monitor all roads at once has made it necessary to find an automated solution. Statistics on accidents caused by potholes show that it is extremely dangerous to drivers and must be urgently resolved. By improving the roads, there can be benefits such as increased tourism, better driving experience, fewer damages to vehicles, and reduced risk of accidents. The project aims to create a prototype system that detects cracks and potholes in roads and alerts officials for immediate repairs. The system will use ESP-32, a WiFi Module, a Camera, a car chassis, and a telebot for transmitting messages to officials.

ACKNOWLEDGEMENT

We wish to express our sincere thanks and deep sense of gratitude to our project guide, **Dr. Berlin Hency**, Associate Professor, School of Electronics Engineering, for her consistent encouragement and valuable guidance offered to us in a pleasant manner throughout the course of the project work.

We are extremely grateful to **Dr. Susan Elias**, Dean of the School of Electronics Engineering, VIT Chennai, for extending the facilities of the School towards our project and for her support.

We express our thanks to our Head of the Department **Dr. Mohanaprasad .K** for his support throughout the course of this project.

We also take this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

K S VIKASH



SIDDHARTH R



SURJYABHO DEB



TABLE OF CONTENTS

Ch.No			TOPIC	PAGE NO.
			ABSTRACT	3
			ACKNOWLEDGEMENT	4
1			INTRODUCTION	7
	1.1		OBJECTIVE AND GOALS	8
	1.2		APPLICATIONS	8
	1.3		FEATURES	8
2			LITERATURE REVIEW	9
3			DESIGN AND IMPLEMENTATION	15
	3.1		COMPONENTS USED	15
		3.1.1	ESP-32 CAM	15
		3.1.2	Arduino Uno	17
	3.2		PROPOSED ARCHITECTURE	18
		3.2.1	ARCHITECTURE	18
		3.2.2	BLOCK DIAGRAM	18
		3.2.3	DESIGN APPROACH	19
		3.2.4	HARDWARE ANALYSIS	19
	3.3		SOFTWARE SETUP	20
		3.3.1	DATASET	20
		3.3.2	MODEL ARCHITECTURE	21
		3.3.3	SOURCE CODE	22
4			CODE	23
	4.1		HARWARE CODE	23
	4.2		CRACK SEGMENTATION CODE	32
5			RESULT AND ANALYSIS	34
	5.1		CRACK SEGMENTATION IMAGES	34

	5.2		TELEGRAM BOT IMAGES	43
	5.3		HARDWARE IMAGES	44
6			CONCLUSION AND FUTURE SCOPE	45
	6.1		CONCLUSION AND INFERENCE	45
	6.2		FUTURE SCOPE AND ENHANCEMENT	45
7			REFERENCES	46
8			BIODATA	47

CHAPTER 1

INTRODUCTION

The roads in India are well known for their horrible conditions due to deterioration by natural and man made causes. This has been posing serious problems for transportation facilities in the matter of safety, time taken to travel and increasing maintenance cost for vehicles. The government cannot monitor every road at a given time practically, hence making it necessary to come up with an automated solution.

“The ministry of road transport and highways said that as many as 4,775 accidents occurred in 2019 due to potholes and 3,564 such accidents were reported in 2020” - Times of India. Statistically, we are able to see that many road accidents these days are caused due to such cracks and potholes. Not only this, according to surveys taken, most drivers are unsatisfied with their on-road experience due to bad road conditions in most places in India. Solving this problem can lead to:

1. Better infrastructure in India which lead to higher rates of Tourism and traveling in the country
2. Better user/driver experience
3. Less damages to cars and vehicles in general
4. Lower risk of accidents on roads
5. Better drainage and water flow on roads

In this project we aim to create a prototype of a system where we detect cracks and potholes in the road and alert the concerned officials in order to fix the road as soon as possible. We will be using ESP-32 which has a WiFi Module, a Camera, a car chassis, and a telebot to transmit the messages to the officials.

1.1 OBJECTIVE AND GOALS

The main objective of this project is to create an automated quad wheeled robot which can detect cracks on the roads and send reports on the crack to the respective authority through telegram. To implement this, the project can be split into three parts: the hardware part which sets up the communication, the telegram bot creation and finally the deep learning part which runs the crack segmentation algorithm.

1.2 APPLICATIONS

This project can be used on roads with not much human movement (roads in hilly regions or through forests). They can also be used as a part of an extended project which is a road laying machine which will lay down the tar only when the crack is detected.

1.3 FEATURES

- Crack Segmentation using a model derived from VGG-16 architecture.
- ESP32 cam which detects the crack on the road and displays it to the local host.
- Telegram bot which gets the alert message from the esp32 to display the crack status to the user.

CHAPTER 2

LITERATURE REVIEW

Author	Title	Year of Publication	Methodology	Inference of the paper
Jihong Liu; Jiaxin Gu; Shan Luo	Research on Road Crack Detection based on Machine Vision	2022	Firstly, the collected crack image is processed by graying, smoothing and grayscale stretching. Then threshold segmentation is carried out, and the background noise is removed to extract the crack target. Finally, according to the statistical curve change trend of the number of crack target pixels from different directions for different types of cracks, the type of cracks is judged; The length of the crack is calculated by extracting and refining the skeleton of the crack, and the average width, maximum and minimum width of the crack are calculated. The area is calculated by the minimum circumscribed rectangle. The test results of the detection system show that the accuracy of crack detection is basically	In view of the low efficiency and low accuracy of manual detection of road cracks, a road crack detection method based on machine vision is studied. The image processing algorithm is applied to crack detection. Through computer simulation, the classification of cracks and the extraction of feature information are realized, and the detection result data are obtained. Collect a large number of sample images for detection experiments, and evaluate the damage degree of cracks according to the standards, so as to provide reference for timely maintenance of the highway. According to the measured data and calculated data, the detection accuracy of this method is basically more than 90%, and the judgment of crack

			more than 90%, the accuracy of crack area, length and average width are more than 95%, and the accuracy of crack classification is 100%.	shape is also quite accurate. Using machine vision crack detection technology can reduce the workload and improve the detection efficiency and accuracy.
Vishal Mandal; Lan Uong; Yaw Adu-Gyamfi	Automated Road Crack Detection Using Deep Convolutional Neural Networks	2018	In this study, we propose an automated pavement distress analysis system based on the YOLO v2 deep learning framework. The system is trained using 7,240 images acquired from mobile cameras and tested on 1,813 road images. The detection and classification accuracy of the proposed distress analyzer is measured using the average F1 score obtained from the precision and recall values. Successful application of this study can help identify road anomalies in need of urgent repair, thereby facilitating a much better civil infrastructure monitoring system.	In the current study, we implemented a deep learning framework to automatically detect and classify different types of cracks. The F1 score for detection without predicting crack class was found out to be 0.8780 and for classification including crack class was 0.7394. It was observed that the distress analyzer so developed is more accurate in evaluating alligator cracks but struggles with transverse cracks. This might be due to the insufficient number of images with transverse cracks in the training database. The disparity in accurately detecting cracks was mainly due to the incorrect labeling of cracks in the image, lower distinction between the crack and background image and shadow formation.

Hyunwoo Cho; Hyuk-Jin Yoon; Ju-Yeong Jung	Image-Based Crack Detection Using Crack Width Transform (CWT) Algorithm	2018	We therefore propose an edge-based crack detection technique consisting of five steps: crack width transform, aspect ratio filtering, crack region search, hole filling, and relative thresholding. In the first step, crack width transform, opposing edges are identified and classified as crack candidate pixels, and a width map is generated. In the second step, aspect ratio filtering, the width map generated is used to remove noise. In the next two steps, the method searches for and restores missing pixels. In the last step, relative thresholding, residual noise is removed through reclassification of crack regions based on image-adaptive thresholding. The performance of this technique was tested using synthetic and real images.	We proposed an edge-based crack detection technique, which consists of five steps: CWT, aspect ratio filtering, crack region search, hole filling, and relative thresholding. CWT can efficiently extract edge pixels by identifying the opposing edge pixel for any given edge pixel and classifying it as a candidate crack pixel. Aspect ratio filtering is proposed for removing noise and improving accuracy using the width map generated in CWT. Crack region search was proposed for restoring the crack regions removed during aspect ratio filtering. In this process, the removed pixels could be identified and restored by searching the candidate pixels on the crack propagation path, thus improving the true positive rate (TPR) of crack detection. Hole filling restored crack regions that were undetected due to non-extraction of the corresponding edges.
---	---	------	--	---

Sadia Mubashshira ; Md. Mushfiquir Azam; Sk. Md. Masudul Ahsan	An Unsupervised Approach for Road Surface Crack Detection	2020	. So, in order to automate this inspection, an unsupervised approach has been proposed in order to detect the pavement cracks. Therefore, a method has been proposed to detect the road surface domain on the basis of color histogram analysis of pavement surfaces. K-means clustering algorithm followed by Otsu thresholding has been done for segmentation purposes in order to detect cracks on 2D road surface images. The presented algorithm provides a satisfactory result in case of detecting and localizing the crack of an image. It can effectively remove the noise and preserve edges which is very useful to attain an accuracy of good extent.	From the experimental results and analysis, it can be concluded that unsupervised approach performs better result than edge detection approach in crack detection. An important issue is that ground truth has been created manually by us. So, it is not possible for a human being to draw 100% accurate ground truth of an image. As a result, sometimes proposed system output yields a better result than manual ground truth.
--	---	------	--	--

Henrique Oliveira; Paulo Lobato Correia	Road surface crack detection: Improved segmentation with pixel-based refinement	2017	The proposed crack segmentation algorithm includes two stages: (i) selection of prominent "crack seeds", adopting an efficient segmentation procedure, after appropriate image smoothing, minimizing the detection of false positives; (ii) iterative binary pixel classification, into the crack or non-crack classes, extending the "seeds" to identify the complete crack shape. The paper also tests the combination of the proposed two stage crack segmentation with three smoothing techniques, to evaluate their suitability for crack detection. As a final step the system classifies the identified cracks as longitudinal, transversal or miscellaneous types. Tests performed with images acquired from different types of sensors (active and non-active), show improved crack segmentation results.	This paper proposed a novel crack segmentation refinement algorithm, which is able to follow complex cracks including several thin branches or presenting a long linear development. The system performs well with the three different preprocessing strategies tested, eventually favoring the simple morphological filtering. Both quantitative and qualitative performance evaluations indicate improved segmentation results.
---	--	------	---	--

Mingxin Nie; Cheng Wang	Pavement Crack Detection based on yolo v3	2019	In recent years, with the rapid development of the economy, road construction has entered the stage of coexistence of construction and conservation. Even road maintenance has become a major aspect of road construction. The government invests a lot of money in road maintenance every year. Therefore, it's very important to detect cracks on the road surface in order to reduce the cost of maintenance. Aiming at the problems of poor real-time performance and low accuracy of traditional pavement crack detection, and using the advantages of deep learning network in target detection, a method based on yolo v3 for pavement crack detection is designed. The collected pictures are manually marked, and the network model is obtained through yolo v3 network training. Finally, the cracks are detected and verified by the obtained model.	In this paper, a neural network-based pavement crack detection model is trained. The traditional pavement crack detection method has low detection accuracy, complicated processing pictures and poor real-time performance. The current popular target recognition algorithm yolo v3 is used for pavement crack identification and compared with other methods. The experimental results show that the detection model can quickly identify the pavement cracks and meet the real-time requirements. It also has a good effect in identifying accuracy. However, due to the diversity of cracks and other reasons, the model has a certain rate of missed detection. In the future work, the data set will be further improved to ensure the completeness of various crack samples and further improve the crack detection model.
-------------------------------	---	------	--	---

CHAPTER 3

DESIGN AND IMPLEMENTATION

3.1 COMPONENTS USED

3.1.1. ESP32-CAM

With the ESP32-S chip, the ESP32-CAM is a very compact camera module. A microSD card slot is included in addition to the OV2640 camera and multiple GPIOs for connecting peripherals. This slot can be used to store images captured by the camera or files that will be provided to clients.

The ESP32-CAM doesn't come with a USB connector, so you need an FTDI programmer to upload code through the U0R and U0T pins (serial pins).

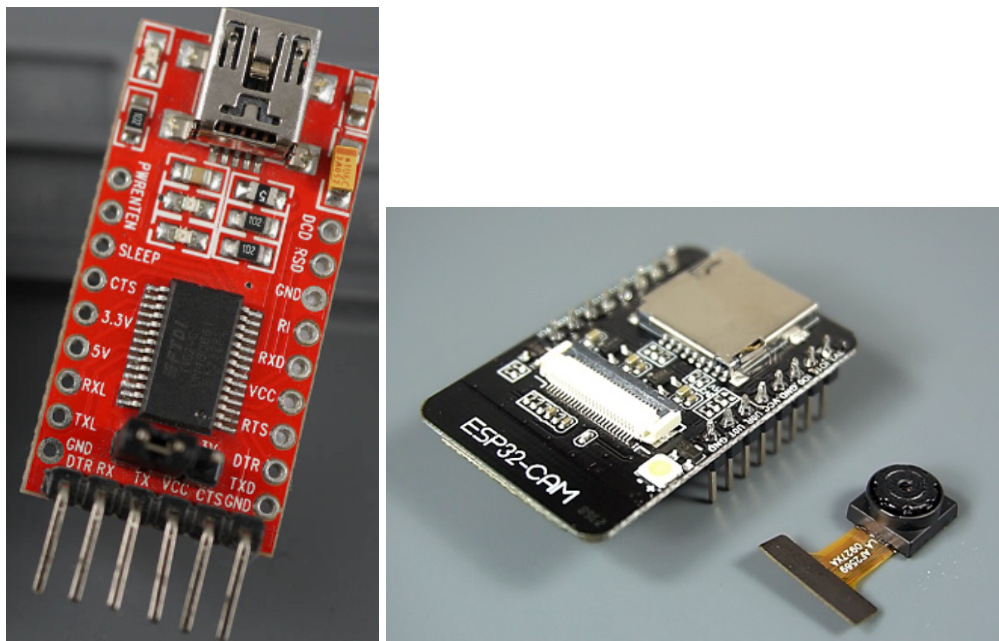
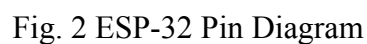


Fig. 1 ESP-32 Cam



GPIO1 and GPIO3 are the serial pins. You need these pins to upload code to your board. Additionally, GPIO 0 also plays an important role, since it determines whether the ESP32 is in flashing mode or not. When GPIO 0 is connected to GND, the ESP32 is in flashing mode.

- GPIO 14: CLK
- GPIO 15: CMD
- GPIO 2: Data 0
- GPIO 4: Data 1 (also connected to the on-board LED)
- GPIO 12: Data 2
- GPIO 13: Data 3

3.1.2. ARDUINO UNO

Arduino UNO is a microcontroller board based on the **ATmega328P**. It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started. You can tinker with your UNO without worrying too much about doing something wrong, worst case scenario you can replace the chip for a few dollars and start over again.



Replaceable chip

The ATmega328P can easily be replaced, as it is not soldered to the board.



EEPROM

The ATmega328P also features 1kb of EEPROM, a memory which is not erased when powered off.



Battery Connector

The Arduino UNO features a barrel plug connector, that works great with a standard 9V battery.

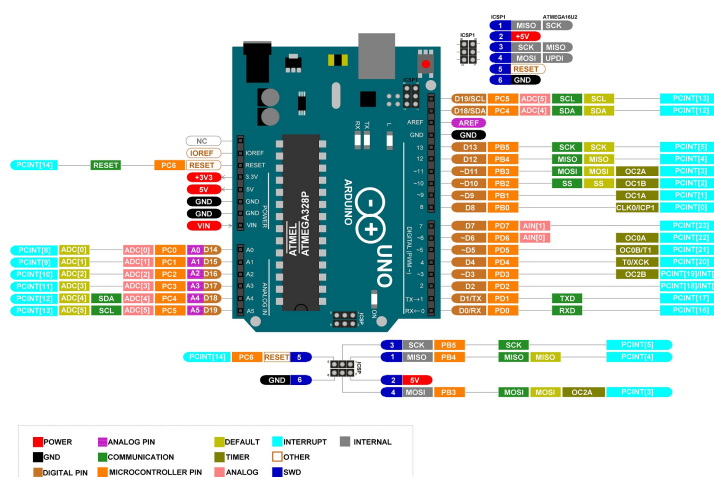


Fig. 3 Arduino UNO Pin Diagram

3.2 PROPOSED SYSTEM

3.2.1. ARCHITECTURE

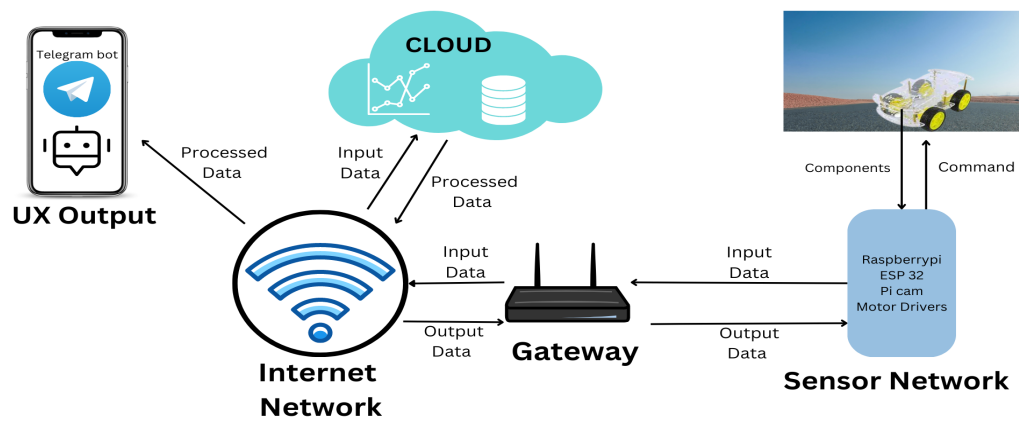


Fig. 4 Proposed IoT Architecture

3.2.2. BLOCK DIAGRAM

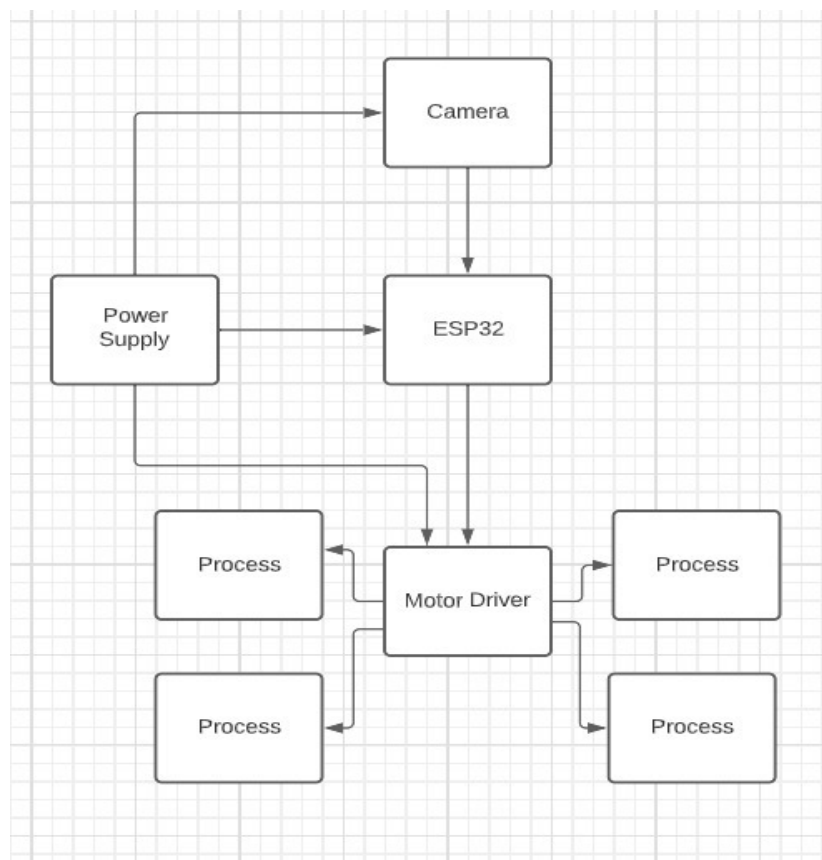


Fig. 5 Flow Diagram of Hardware Working

3.2.3. DESIGN APPROACH

We are using an ESP32 Camera module for capturing images of the road. The images are captured using the ESP32 Cam fitted on a car with 4 motors that can run along the road and capture the images. The car moves forward for 5 secs then it stops and captures a picture.

The picture that is captured is sent to a telegram bot. The images in the telegram bot can be accessed by everyone from anywhere. The captured images sent to the telegram bot is then called through an API and fed to the model for crack detection and analysis.

3.2.4. HARDWARE ANALYSIS

The hardware consists of an ESP-32 Camera module along with an Arduino Uno, L298N motor driver, 4 DC motors and a chasis. The ESP32 is connected to the Arduino Uno board for constant power supply of 5V. The pins of the ESP32 are connected to the motor driver. The motor driver is intern connected to a 12V battery for power. The pin connectections are as per the following table:

ESP32 Cam Pins	L298N Driver pins
IO12	enA
IO13	IN1
IO15	IN2
IO14	IN3
IO2	IN4
IO12	enB
5V	5V
GND	GND

Fig. 6 Hardware Pin Connections

The diagram of the hardware components is as follows:

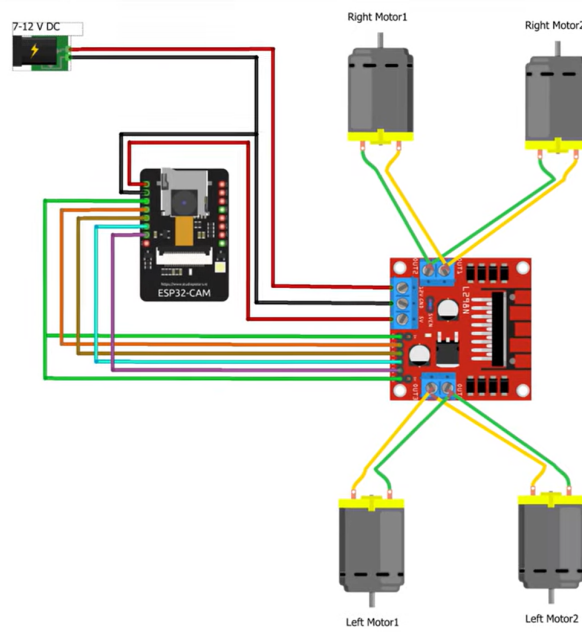


Fig. 7 Hardware Circuit Diagram

3.3. SOFTWARE SETUP

3.3.1 DATASET

The dataset used for implementing crack segmentation is the DeepCrack dataset. It is a public benchmark dataset with cracks in multiple scales and scenes to evaluate the crack detection systems. The dataset is split into four folders for the implementation of our code: train images, train labels, test images and test labels. These directories are available in the above mentioned format while downloading the dataset.

3.3.2 MODEL ARCHITECTURE

For crack segmentation, a binary image formulation is used as the segments detect ‘0’ for non-crack and ‘1’ for crack. The model architecture is based on VGG-16, but with a slight modification. The first 13 layers correspond to that of VGG-16 and the fully

connected layer and the pooling layers are discarded (as seen in Figure 1). This discarding is done due to two reasons:

- The layer after the 5th pooling plane yields an output plane which is very small and the result is quite fuzzy.
- The fully connected layers are computationally intensive, which is memory/time consuming.

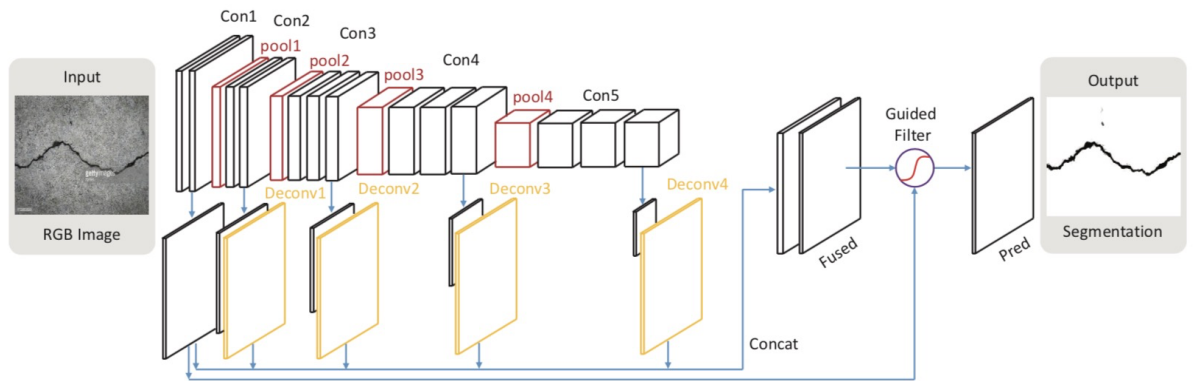


Fig. 8 Proposed Deep Learning Algorithm Architecture

3.3.3 SOURCE CODE

The source code for the DeepCrack dataset was found in the repository of DeepSegmentor, which also contains the code for RoadNet road detection technology based on satellite images. The repository was well organized as training and testing commands of the dataset were reduced to running one shell command.

After setting up the dataset in the methodology as mentioned above, the model can be trained using the following command:

```
sh ./scripts/train_deepcrack.sh <gpu_id>
```

Here the `gpu_id` was set to -1 as the training was done in a nps machine. The trained model was then stored in the checkpoints directory. Using this trained model path, the testing code can be run as follows:

```
sh ./scripts/test_deepcrack.sh <gpu_id>
```

The tested results will automatically store in the results folder and they'll be of image format. Each image from the testing dataset will result in 1 fused image and 5 masked images each with a different side.

CHAPTER 4

CODE

4.1. HARDWARE CODE

```
#include <Arduino.h>

#include <WiFi.h>

#include <WiFiClientSecure.h>

#include "soc/soc.h"

#include "soc/rtc_cntl_reg.h"

#include "esp_camera.h"

#include <UniversalTelegramBot.h>

#include <ArduinoJson.h>


struct MOTOR_PINS

{

    int pinEn;

    int pinIN1;

    int pinIN2;

};


std::vector<MOTOR_PINS> motorPins =

{

    {12, 13, 15}, //RIGHT_MOTOR Pins (EnA, IN1, IN2)

    {12, 14, 2}, //LEFT_MOTOR Pins (EnB, IN3, IN4)

};


#define RIGHT_MOTOR 0

#define LEFT_MOTOR 1
```

```

#define FORWARD 1

#define BACKWARD -1

#define STOP 0


const char* ssid = "Samsung";

const char* password = "fenn8298";


// Initialize Telegram BOT

String BOTtoken = "6265399298:AAGCTkx35uiz8coQcgEm93daQ9sqUNxNKpY"; // your Bot
Token (Get from Botfather)


// Use @myidbot to find out the chat ID of an individual or a group

// Also note that you need to click "start" on a bot before it can

// message you

String CHAT_ID = "1535280260";


bool sendPhoto = false;


WiFiClientSecure clientTCP;

UniversalTelegramBot bot(BOTtoken, clientTCP);


#define FLASH_LED_PIN 4

bool flashState = LOW;


//Checks for new messages every 1 second.

int botRequestDelay = 1000;

unsigned long lastTimeBotRan;

```

```
//CAMERA_MODEL_AI_THINKER
```

```
#define PWDN_GPIO_NUM 32
```

```
#define RESET_GPIO_NUM -1
```

```
#define XCLK_GPIO_NUM 0
```

```
#define SIOD_GPIO_NUM 26
```

```
#define SIOC_GPIO_NUM 27
```

```
#define Y9_GPIO_NUM 35
```

```
#define Y8_GPIO_NUM 34
```

```
#define Y7_GPIO_NUM 39
```

```
#define Y6_GPIO_NUM 36
```

```
#define Y5_GPIO_NUM 21
```

```
#define Y4_GPIO_NUM 19
```

```
#define Y3_GPIO_NUM 18
```

```
#define Y2_GPIO_NUM 5
```

```
#define VSYNC_GPIO_NUM 25
```

```
#define HREF_GPIO_NUM 23
```

```
#define PCLK_GPIO_NUM 22
```

```
void configInitCamera(){
```

```
    camera_config_t config;
```

```
    config.ledc_channel = LEDC_CHANNEL_0;
```

```
    config.ledc_timer = LEDC_TIMER_0;
```

```
    config.pin_d0 = Y2_GPIO_NUM;
```

```
    config.pin_d1 = Y3_GPIO_NUM;
```

```
    config.pin_d2 = Y4_GPIO_NUM;
```

```
    config.pin_d3 = Y5_GPIO_NUM;
```

```
    config.pin_d4 = Y6_GPIO_NUM;
```

```

config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;

//init with high specs to pre-allocate larger buffers
if(psramFound()){
    config.frame_size = FRAMESIZE_UXGA;
    config.jpeg_quality = 10; //0-63 lower number means higher quality
    config.fb_count = 2;
} else {
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 12; //0-63 lower number means higher quality
    config.fb_count = 1;
}

// camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {

```

```

Serial.printf("Camera init failed with error 0x%x", err);

delay(1000);

ESP.restart();
}

// Drop down frame size for higher initial frame rate

sensor_t * s = esp_camera_sensor_get();

s->set_framesize(s, FRAMESIZE_CIF); //
UXGA|SXGA|XGA|SVGA|VGA|CIF|QVGA|HQVGA|QQVGA
}

String sendPhotoTelegram() {
    const char* myDomain = "api.telegram.org";

    String getAll = "";

    String getBody = "";

    camera_fb_t * fb = NULL;

    fb = esp_camera_fb_get();

    if(!fb) {
        Serial.println("Camera capture failed");

        delay(1000);

        ESP.restart();

        return "Camera capture failed";
    }

    Serial.println("Connect to " + String(myDomain));

    if (clientTCP.connect(myDomain, 443)) {
        Serial.println("Connection successful");
    }
}

```

```
String head = "--RandomNerdTutorials\r\nContent-Disposition: form-data; name=\"chat_id\";\r\n\r\n" + CHAT_ID + "\r\n--RandomNerdTutorials\r\nContent-Disposition: form-data; name=\"photo\"; filename=\"esp32-cam.jpg\"\r\nContent-Type: image/jpeg\r\n\r\n";
```

```
String tail = "\r\n--RandomNerdTutorials--\r\n";
```

```
uint16_t imageLen = fb->len;
```

```
uint16_t extraLen = head.length() + tail.length();
```

```
uint16_t totalLen = imageLen + extraLen;
```

```
clientTCP.println("POST /bot"+BOTtoken+"/sendPhoto HTTP/1.1");
```

```
clientTCP.println("Host: " + String(myDomain));
```

```
clientTCP.println("Content-Length: " + String(totalLen));
```

```
clientTCP.println("Content-Type: multipart/form-data; boundary=RandomNerdTutorials");
```

```
clientTCP.println();
```

```
clientTCP.print(head);
```

```
uint8_t *fbBuf = fb->buf;
```

```
size_t fbLen = fb->len;
```

```
for (size_t n=0;n<fbLen;n=n+1024) {
```

```
    if (n+1024<fbLen) {
```

```
        clientTCP.write(fbBuf, 1024);
```

```
        fbBuf += 1024;
```

```
    }
```

```
    else if (fbLen%1024>0) {
```

```
        size_t remainder = fbLen%1024;
```

```
        clientTCP.write(fbBuf, remainder);
```

```
    }
```

```
}
```

```

clientTCP.print(tail);

esp_camera_fb_return(fb);

int waitTime = 10000; // timeout 10 seconds
long startTimer = millis();
boolean state = false;

while ((startTimer + waitTime) > millis()){
    Serial.print(".");
    delay(100);
    while (clientTCP.available()) {
        char c = clientTCP.read();
        if (state==true) getBody += String(c);
        if (c == '\n') {
            if (getAll.length()==0) state=true;
            getAll = "";
        }
        else if (c != '\r')
            getAll += String(c);
        startTimer = millis();
    }
    if (getBody.length()>0) break;
}
clientTCP.stop();
Serial.println(getBody);
}

```

```

else {

    getBody="Connected to api.telegram.org failed.";

    Serial.println("Connected to api.telegram.org failed.");

}

return getBody;

}

void setUpPinModes()

{

    for (int i = 0; i < motorPins.size(); i++)

    {

        pinMode(motorPins[i].pinEn, OUTPUT);

        pinMode(motorPins[i].pinIN1, OUTPUT);

        pinMode(motorPins[i].pinIN2, OUTPUT);

    }

    digitalWrite(motorPins[RIGHT_MOTOR].pinIN1, LOW);

    digitalWrite(motorPins[LEFT_MOTOR].pinIN2, LOW);

}

void setup(){

    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

    // Init Serial Monitor

    Serial.begin(115200);

    setUpPinModes();

    // Set LED Flash as output

    pinMode(FLASH_LED_PIN, OUTPUT);

```

```

digitalWrite(FLASH_LED_PIN, flashState);

// Config and init the camera
configInitCamera();

// Connect to Wi-Fi
WiFi.mode(WIFI_STA);
Serial.println();
Serial.print("Connecting to ");
Serial.println(ssid);
WiFi.begin(ssid, password);

clientTCP.setCACert(TELEGRAM_CERTIFICATE_ROOT); // Add root certificate for
api.telegram.org

while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
}
Serial.println();
Serial.print("ESP32-CAM IP Address: ");
Serial.println(WiFi.localIP());
}

void loop() {
    digitalWrite(motorPins[RIGHT_MOTOR].pinIN1, LOW);
    digitalWrite(motorPins[LEFT_MOTOR].pinIN2, LOW);
    Serial.println("Preparing photo");
    sendPhotoTelegram();
    delay(5000);
    digitalWrite(motorPins[RIGHT_MOTOR].pinIN1, HIGH);

```

```

digitalWrite(motorPins[LEFT_MOTOR].pinIN2, HIGH);

delay(5000);

digitalWrite(motorPins[RIGHT_MOTOR].pinIN1, LOW);

digitalWrite(motorPins[LEFT_MOTOR].pinIN2, LOW);

}

```

4.2. CRACK SEGMENTATION CODE

```

import os
from options.test_options import TestOptions
from data import create_dataset
from models import create_model
from util.visualizer import save_images
from util import html

from typing import Protocol, runtime_checkable

@runtime_checkable
class MyProtocol(Protocol):
    def my_method(self):
        pass

if __name__ == '__main__':
    opt = TestOptions().parse() # get test options
    # hard-code some parameters for test
    opt.num_threads = 1 # test code only supports num_threads = 1
    opt.batch_size = 1 # test code only supports batch_size = 1
    opt.serial_batches = True # disable data shuffling; comment this line if results on randomly chosen
images are needed.
    opt.no_flip = True # no flip; comment this line if results on flipped images are needed.
    opt.display_id = -1 # no visdom display; the test code saves the results to a HTML file.
    dataset = create_dataset(opt) # create a dataset given opt.dataset_mode and other options
    model = create_model(opt) # create a model given opt.model and other options
    model.setup(opt) # regular setup: load and print networks; create schedulers
    # create a website
    web_dir = os.path.join(opt.results_dir, opt.name, '%s_%s' % (opt.phase, opt.epoch)) # define the
website directory
    webpage = html.HTML(web_dir, 'Experiment = %s, Phase = %s, Epoch = %s' % (opt.name,
opt.phase, opt.epoch))
    # test with eval mode. This only affects layers like batchnorm and dropout.
    if opt.eval:
        model.eval()
    for i, data in enumerate(dataset):
        if i >= opt.num_test: # only apply our model to opt.num_test images.
            break
        model.set_input(data) # unpack data from data loader

```

```
model.test()          # run inference
visuals = model.get_current_visuals() # get image results
img_path = model.get_image_paths()    # get image paths
if i % 5 == 0: # save images to an HTML file
    print('processing (%04d)-th image... %s' % (i, img_path))
    save_images(webpage, visuals, img_path, aspect_ratio=opt.aspect_ratio,
width=opt.display_winsize)
webpage.save() # save the HTML
```

CHAPTER 5

RESULTS AND OUTPUTS

5.1. CRACK SEGMENTATION IMAGES



Fig. 9 Test Sample of Road Crack

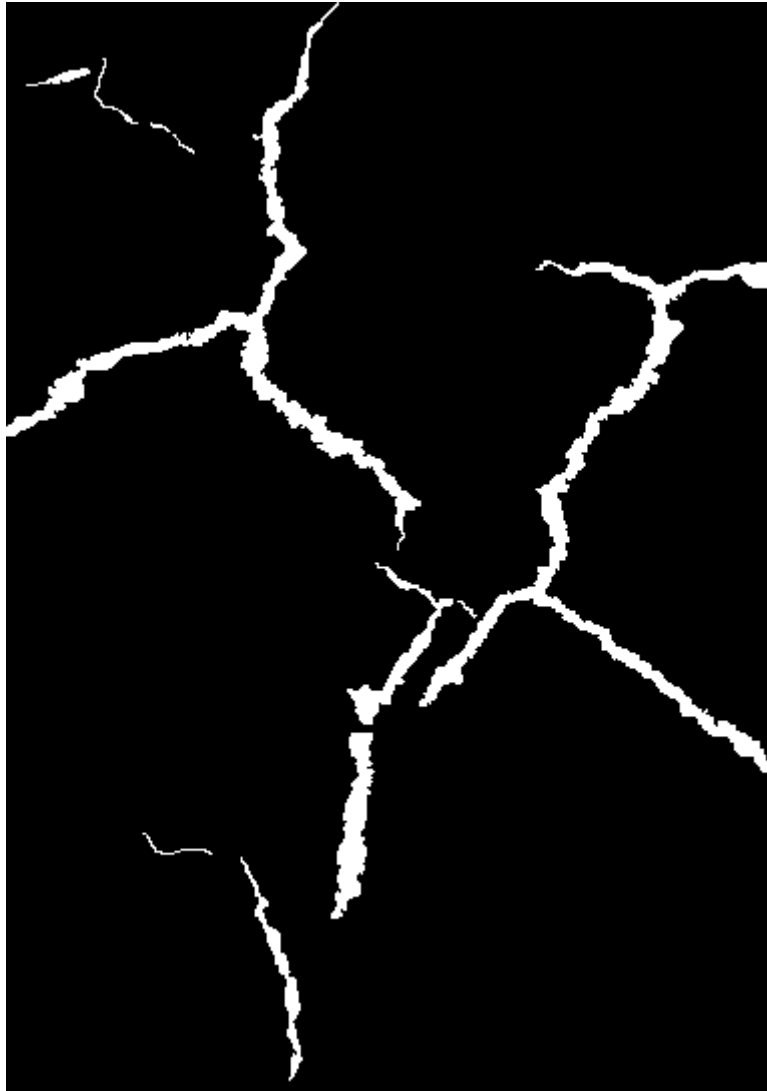


Fig. 10 Grayscale Image with Labels

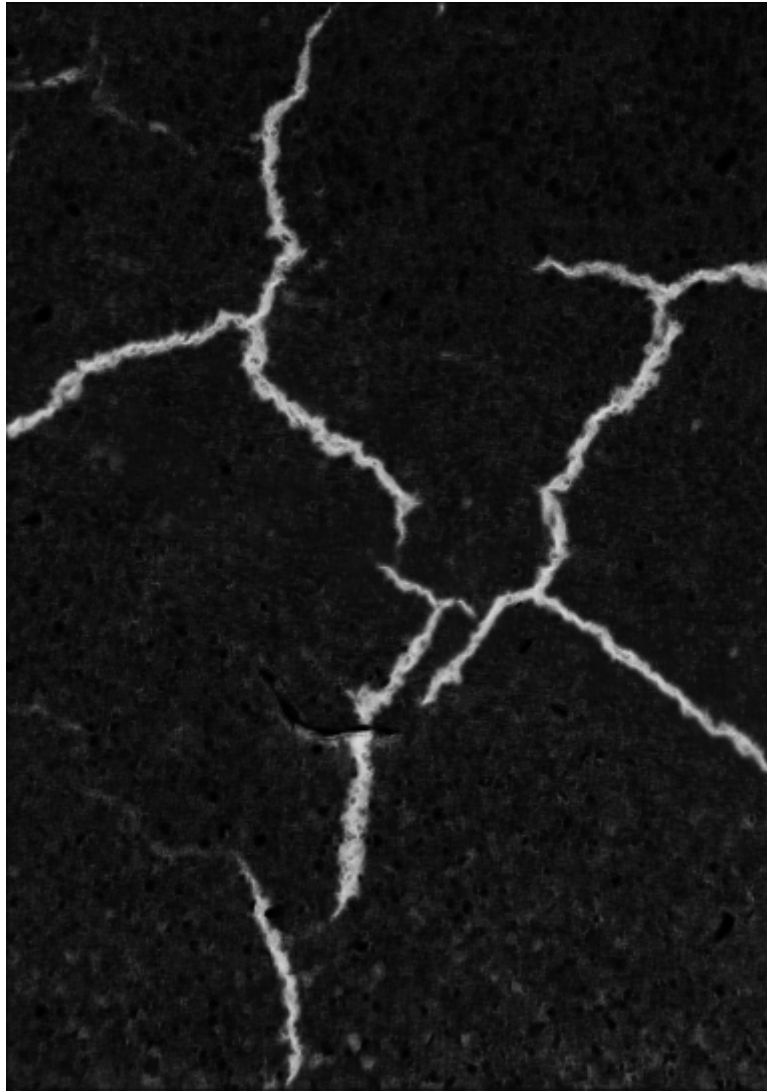


Fig. 11 Image Side 1

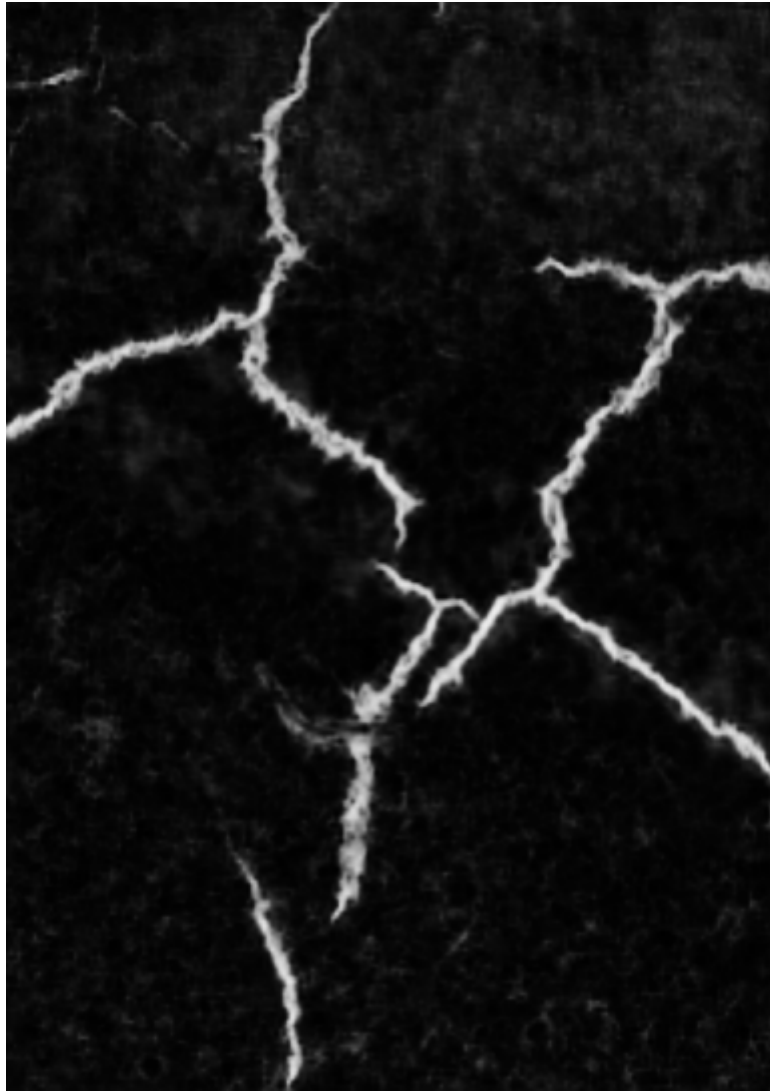


Fig. 12 Image Side 2

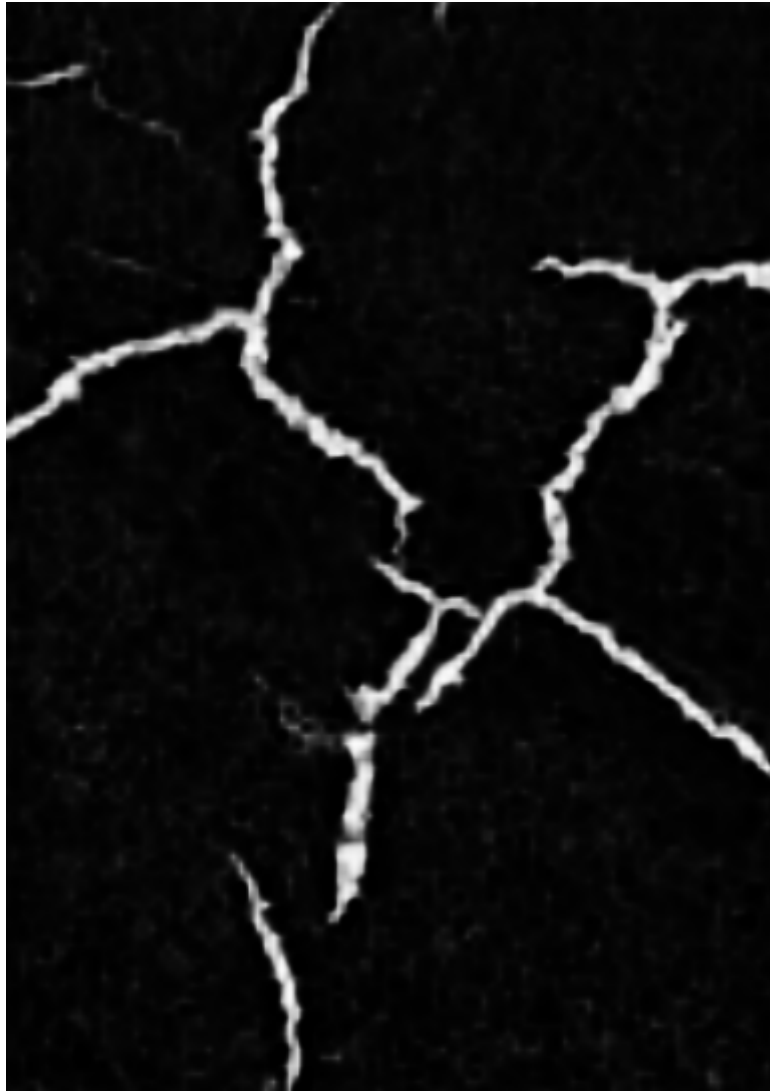


Fig. 13 Image Side 3



Fig. 14 Image Side 4



Fig. 15 Image Side 5

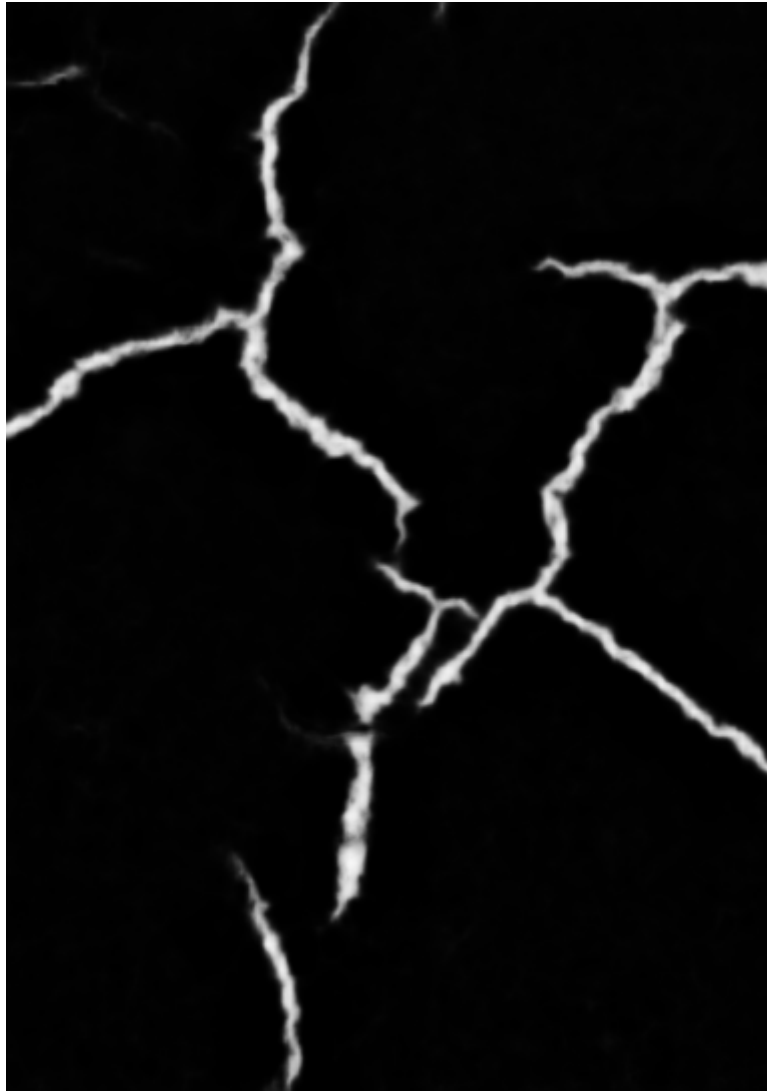


Fig. 16 Fused of Image after Crack Detection

The above images are obtained from the results folder after running the inference of the model. The tested sample outputs are converted gray scale images.

5.2. TELEGRAM BOT IMAGES



Fig. 17 Telegram Bot with Road Crack Images

5.2. HARDWARE IMAGES

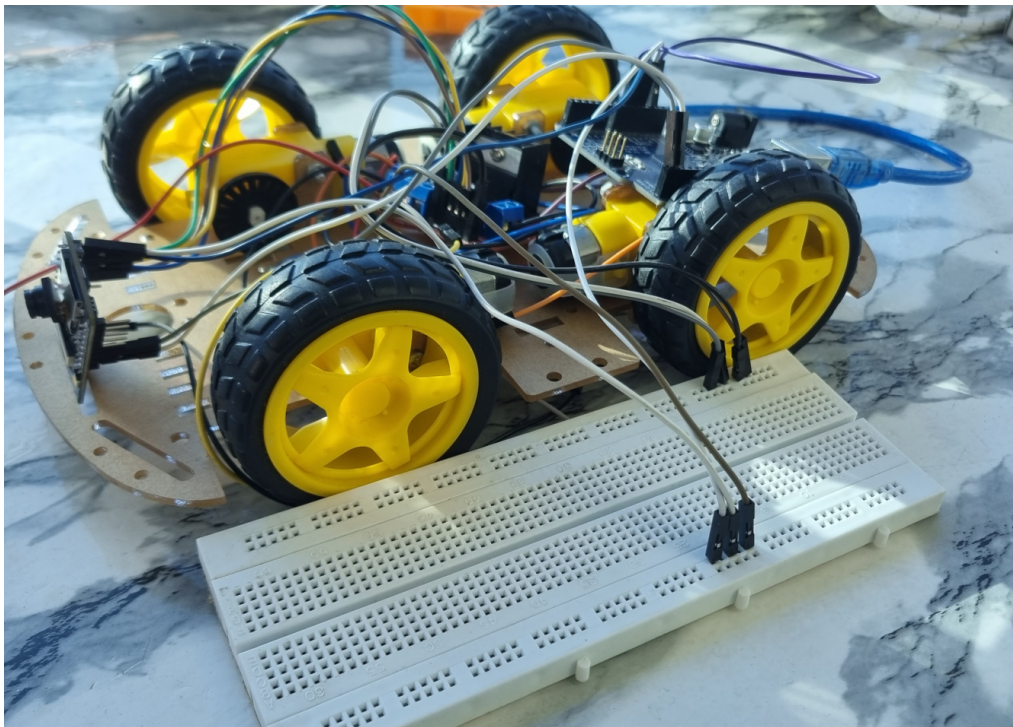
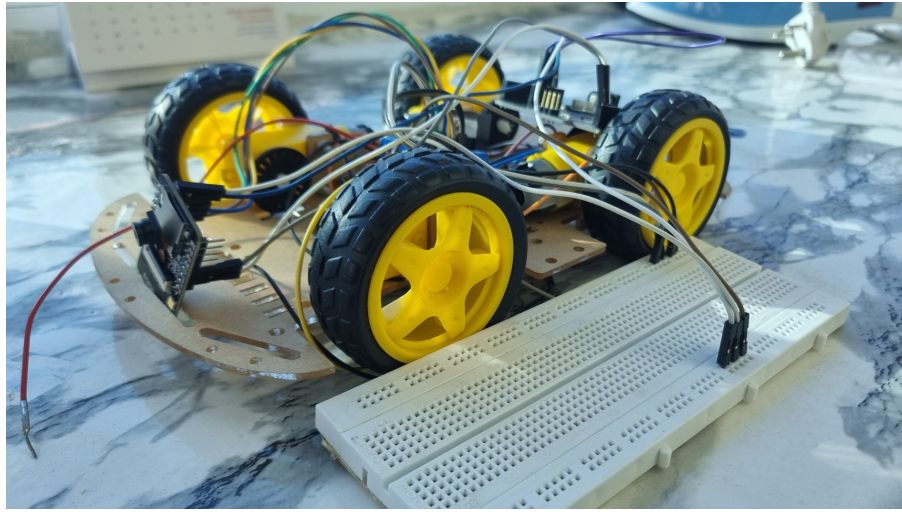


Fig. 18 Hardware Robot Car for capturing road cracks

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION AND INFERENCE

The car with ESP32 camera module was made which could be controlled by the local host server and the live feed from the camera was fed to the local host. Images captured from this feed was used to run the crack segmentation and detect cracks.

6.2 FUTURE SCOPE AND ENHANCEMENT

There exists a tremendous scope for this project and can be developed into a real-life solution for road maintenance. The car can be integrated with machine learning models to have obstacle detection and stop when a crack is detected. The car can also be made completely autonomous with a GPS to navigate the roads of India and understand it's surroundings with LIDAR/RADAR point-cloud based object detection. The project can also implement a real-time video feedback from which the car detects, processes and passes the cracks found on the road, to the model continuously. Furthermore, various deep learning algorithms can be explored and compared to provide the best accuracy and a new architecture that is a combination of those algorithms can be proposed.

CHAPTER 7

REFERENCES

- [1] J. Liu, J. Gu and S. Luo, "Research on Road Crack Detection based on Machine Vision," 2022 IEEE 6th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC), Beijing, China, 2022, pp. 543-547, doi: 10.1109/IAEAC54830.2022.9929645.
- [2] V. Mandal, L. Uong and Y. Adu-Gyamfi, "Automated Road Crack Detection Using Deep Convolutional Neural Networks," 2018 IEEE International Conference on Big Data (Big Data), Seattle, WA, USA, 2018, pp. 5212-5215, doi: 10.1109/BigData.2018.8622327.
- [3] H. Cho, H. -J. Yoon and J. -Y. Jung, "Image-Based Crack Detection Using Crack Width Transform (CWT) Algorithm," in IEEE Access, vol. 6, pp. 60100-60114, 2018, doi: 10.1109/ACCESS.2018.2875889.
- [4] S. Mubashshira, M. M. Azam and S. M. Masudul Ahsan, "An Unsupervised Approach for Road Surface Crack Detection," 2020 IEEE Region 10 Symposium (TENSYP), Dhaka, Bangladesh, 2020, pp. 1596-1599, doi: 10.1109/TENSYP50017.2020.9231023.
- [5] H. Oliveira and P. L. Correia, "Road surface crack detection: Improved segmentation with pixel-based refinement," 2017 25th European Signal Processing Conference (EUSIPCO), Kos, Greece, 2017, pp. 2026-2030, doi: 10.23919/EUSIPCO.2017.8081565.
- [6] M. Nie and C. Wang, "Pavement Crack Detection based on yolo v3," 2019 2nd International Conference on Safety Produce Informatization (IICSPI), Chongqing, China, 2019, pp. 327-330, doi: 10.1109/IICSPI48186.2019.9095956.
- [7] Liu, Yahui & Yao, Jian & Lu, Xiaohu & Xie, Renping & Li, Li. (2019). DeepCrack: A Deep Hierarchical Feature Learning Architecture for Crack Segmentation. Neurocomputing. 338. 139-153. 10.1016/j.neucom.2019.01.036.
- [8] Liu, Yahui & Yao, Jian & Lu, Xiaohu & Xia, Menghan & Wang, Xingbo & Liu, Yuan. (2018). RoadNet: Learning to Comprehensively Analyze Road Networks in Complex Urban Scenes From High-Resolution Remotely Sensed Images. IEEE Transactions on Geoscience and Remote Sensing. 57. 2043-2056. 10.1109/TGRS.2018.2870871.

<https://github.com/yhlleo/DeepSegmentor>

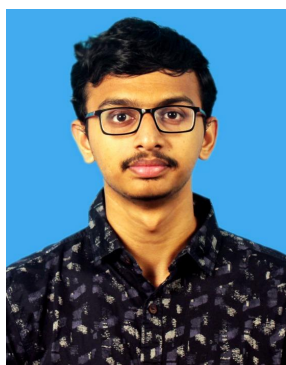
<https://github.com/yhlleo/DeepCrack>

CHAPTER 8

BIODATA



Name: Vikash K S
Email: vikash.ks2020@vitstudent.ac.in
Phone Number: 9840338863
Permanent Address: Jains Anusri Apts, TC, 3rd Floor,
18th Avenue, 86th Street, Ashok Nagar, Chennai-83



Name: Siddharth R
Email: siddharth.ramanathan2020@vitstudent.ac.in
Phone Number: 9940118319
Permanent Address: V, 3rd street, Marvel Rover View
County, Manapakkam, Chennai 600125



Name : Surjyabho Deb
Mobile Number : 9007719749
E-mail: surjyabho.deb2020@vitstudent.ac.in
Permanent Address : 2/1 Dover Terrace, Kolkata
700019